

TEACHING GUIDE · MBOT2 ROVER ROBOTICS PROGRAM

Grade 9 Engineering & Autonomy

Robots That Work

20

SESSIONS

1 h

EACH

mBot2 Ranger

TANK + ROBOTIC ARM

Full Python

CODING MODE

★ **Final project: Robots at Work**

An autonomous mission simulating a real job: navigate by line, detect a marker, pick an object with the arm and deliver it — pitched and scored at Demo Day.

Grade 9 at a Glance

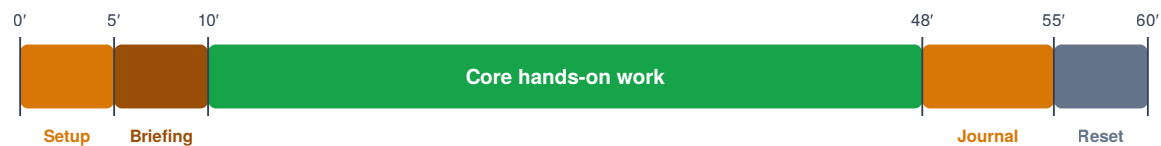
Learning objectives — by the end of the year, students can:

- Write, structure and review Python: functions, constants, state machines
- Implement and tune proportional control; argue with graphs
- Build the Ranger arm; program safe, reliable pick & place
- Run an engineering project: spec, sprints, reviews, recovery
- Deliver an autonomous mission and pitch it (Robots at Work)

The 20 sessions, colored by unit * in session titles = buffer session (tolerates compression if the schedule slips)



Every session runs on the same rhythm



Non-negotiables for 1-hour sessions: robots stay assembled between sessions on a labeled cart shelf · code saved to mBlock cloud accounts every session · battery rotation with a per-team battery manager · extension cards at stations so fast teams never wait.

Known friction points this year: the Ranger build guide is digital-only — download before Session 8 · never force the arm past resistance (gears strip) · exact Python function names vary by mBlock version: trust autocomplete · rebuild capstone courses from tape-measure photos.

The Boxes — Official Contents Reference

Both boxes are used this year. Session parts panels tag every piece with its source box.

BOX 1 · “mBot2”

CyberPi ×1	Ultrasonic Sensor 2 ×1	M4×8 mm screw ×6
mBot2 Shield ×1 · battery built in	Quad RGB Sensor ×1	M2.5×12 mm screw ×4
Chassis ×1	Motor cable ×2	Line-following track map ×1
Encoder motor ×2	mBuild cable 10 cm ×2	USB cable ×1
Wheel hub ×2	mBuild cable 20 cm ×1	Screwdriver ×1
Slick tyre ×2	M4×25 mm screw ×6	
Mini wheel ×1	M4×14 mm screw ×6	

Quantities from the official packing list. Bluetooth dongle NOT included — Makeblock advises it for older computers (sold separately).

BOX 2 · “Rover Robotics Add-on Pack”

Servo pack ×2 used	Slide beam 2×4 ≥×1	Nut ≥×30
Bluetooth Controller ×1	Slide beam 2×6 ≥×1	Plastic shock absorber spares — unused in build
Track ×2	Plate 3×8 ≥×1	Metal shock absorber ×4 used
Wheel hub ×4 halves used	Shaft collar ×12 used	Long plastic ring ≥×2
Small wheel hub ×2 used	Flange bearing ≥×4	Short plastic ring ≥×8
D shaft ≥×6	Socket wrench ×1	Rubber band ≥×6
Bracket 90° 3×3 ≥×6	Screw M3×6 mm ×12 used	Board 1–8 (body shells) peel paper film
Bracket 120° 3×3 ≥×3	Screw M4×10 mm ≥×30	Quick Start Guide ×1
U bracket ≥×4	Screw M4×22 mm ≥×14	
T plate ×2 used	Screw M4×30 mm ≥×8	

As printed on the box (no quantities given). Online docs: search “mBot2 Rover” at support.makeblock.com.

Same Robot, New Language

Goal of this hour: Every student writes and runs a Python program that drives a square — in pairs.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
from Grade 8

Rover add-on
box



Laptop + mBlock
mBlock Python
mode

Classroom



Engineering
journal
error-message notes

Classroom

TEACH IT — THE CONCEPT

Open with the Grade 8 fair photos, then the year's premise: same robot, professional tools. mBlock's Python mode is a real editor — text, autocomplete, error messages. The first program is deliberately familiar: the square. They've built it in blocks (Grade 7) and read it in Python (Grade 8, Session 17); today they type it from a blank file. Familiar problem + new medium = confidence, not overwhelm.

Institute **pair programming** as this year's standard: driver types, navigator reads ahead and catches errors; swap every 10 minutes. It's how professionals work and it halves the syntax frustration of week one. Also set the error-message culture: an error message is information, not an insult — read it aloud, bottom line first.

LESSON FLOW (45' CORE)

- 8'** Editor tour: file, run, stop, console, autocomplete. Break something on purpose; read the error together.
- 20'** Pairs type the square from scratch; run on the robot; swap roles halfway.
- 12'** Extension ladder: triangle → hexagon → any-N polygon (the 360/N insight, now as code).
- 5'** Journal: one error message you met and what it was telling you.

PYTHON (MBLOCK → PYTHON MODE)

```
import cyberpi
import mbot2

# our old friend, typed from scratch
for i in range(4):
    mbot2.straight(30)    # cm forward
    mbot2.turn(90)       # degrees right

cyberpi.display.show_label('done!', 24, 'center')
```

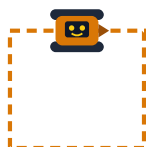
Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- IndentationError and missing colons will dominate today — predict it aloud so nobody feels alone.
- Pairs stay together for the whole Python Bootcamp unit.

EXPECTED RESULT AT THE END OF THIS SESSION



typed Python drives the same square as Grade 7's blocks



- ✓ Square runs from hand-typed Python
- ✓ Pair-programming roles practiced with a swap
- ✓ At least one error message read, decoded, and fixed by the pair

Functions

Goal of this hour: Motion wrapped into reusable, parameterized functions — the vocabulary of clean code.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Engineering
journal
library design notes
Classroom

TEACH IT — THE CONCEPT

A function is a named behavior: define `square(size)` once, call it anywhere, change it in one place. Connect backwards: functions are to code what loops were to repeated blocks in Grade 7 — the next rung on the same ladder (don't repeat yourself). The parameter is the power move: `square(20)`, `square(40)` — one definition, infinite squares.

Build a small personal library today: `square(size)`, `polygon(n, size)`, `celebrate()` (LEDs + sound). This library literally gets imported into the capstone — tell them that now, so “write it well” has a reason. Naming matters: `celebrate()` reads like English; `func2()` reads like regret.

LESSON FLOW (45' CORE)

- 10'** Live-code `square()` together; call it three times with different sizes.
- 15'** Pairs build `polygon(n, size)` — the $360/N$ rule becomes one line of math in code.
- 12'** Design `celebrate()` freely: lights + sound + a wiggle; must be a single call.
- 8'** Library review: swap code with another pair; can they use your functions without asking you?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi

def polygon(n, size):
    for i in range(n):
        mbot2.straight(size)
        mbot2.turn(360 / n)

def celebrate():
    cyberpi.led.on(255, 180, 0)
    cyberpi.audio.play('yay')

polygon(5, 25) # pentagon, one line
celebrate()
```

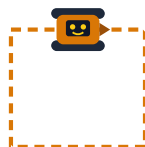
Watch out

- The another-pair-can-use-it test is the real definition of a good function.
- Watch for logic vs. syntax confusion: `polygon(3, ...)` turning 120° is correct — remind them of the exterior angle.

● EXPECTED RESULT AT THE END OF THIS SESSION

```
def polygon(n, size):
    for i in range(n):
        mbot2.straight(size)
        mbot2.turn(360/n)

polygon(4, 30)
```



one function, many shapes — the personal library begins

- ✓ `polygon(n, size)` draws at least three different shapes
- ✓ `celebrate()` runs as a single call
- ✓ Another pair used the library without explanations

Sensing in Python

Goal of this hour: Live sensor values read, displayed and logged — the robot's senses in text form.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Line-following
map
line for the offset
explorer
mBot2 box



Ruler / tape
measure
verify distances
Classroom



Engineering
journal
offset sketch
Classroom

TEACH IT — THE CONCEPT

Sensors in Python return numbers you can hold in a variable: `d = cyberpi.ultrasonic2.get()`. That one line changes the relationship — in blocks, values flowed invisibly; in Python, students capture, print, compare and compute with them. Build the classic diagnostic tool first: a loop printing distance to the screen ten times a second (a live dashboard, and their debugging best friend for the rest of the year).

Then the line sensor: it reports how far off-center the line is (an **offset**, roughly $-100\dots+100$). Have students push the robot along a line by hand and watch the number swing negative–zero–positive. That number IS next week's error signal for P-control; today they just need to feel what it measures.

LESSON FLOW (45' CORE)

- 12'** Build the distance dashboard; verify against a ruler; find sensor limits again (they know the drill).
- 12'** Threshold play: print 'NEAR!' when `d < 15` — an if on a captured value.
- 15'** Line offset explorer: hand-push the robot along a line, watch the offset swing; sketch its meaning.
- 6'** Journal: what does `offset = -60` mean physically? Draw it.

PYTHON (MBLOCK → PYTHON MODE)

```
import cyberpi, mbot2, time

while True:
    d = cyberpi.ultrasonic2.get()
    off =
    cyberpi.quad_rgb_sensor.get_offset_track()
    cyberpi.display.show_label('d=' +
    str(d), 16, 'top')
    cyberpi.display.show_label('off=' +
    str(off), 16, 'center')
    if d < 15:
        cyberpi.display.show_label('NEAR!',
        24, 'bottom')
    time.sleep(0.1)
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- `str(d)` trips everyone — numbers must become text to display; let them hit the error, then explain.
- Calibrate the line sensor first (the Grade 7 ritual) or the offset numbers are noise.

EXPECTED RESULT AT THE END OF THIS SESSION

```
while True:
    d = ultrasonic2.get()
    show('d=' + str(d))
    if d < 15:
        show('NEAR!')
    time.sleep(0.1)
```

```
d = 34 cm
d = 27 cm
d = 12 cm
STOP
```

a live sensor dashboard — the year's debugging companion

- ✓ Dashboard shows live distance + line offset
- ✓ NEAR! fires at the right threshold
- ✓ Team can explain offset's sign and rough magnitude physically

Logic & Refactor

Goal of this hour: Grade 7's obstacle avoidance rebuilt in Python — then refactored into a clean function.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Boxes / obstacles
arena

Classroom



Engineering
journal

before/after snippets

Classroom

TEACH IT — THE CONCEPT

Full control flow: `if / elif / else` and `while`. The vehicle is deliberately nostalgic — rewrite Grade 7's wander mode in Python. They know exactly what it should do, so all attention goes to expressing it in the new language: a `while-True` loop, a distance check, the back-up-and-random-turn escape.

Then introduce **refactoring**: improving code's shape without changing its behavior. Extract the escape maneuver into `avoid()`, name the magic numbers as constants (`SAFE_CM = 12`), and re-run — identical robot behavior, radically clearer code. That “same behavior, better shape” discipline is the heart of code quality, and it's a rubric line in the capstone code review.

LESSON FLOW (45' CORE)

- 15'** Rewrite wander in Python: `while True` + distance check + escape.
- 15'** Refactor: extract `avoid()`, name constants, add two comments that say **WHY** not what.
- 10'** Arena test: does refactored wander behave identically? (It must — that's the definition.)
- 5'** Journal: before/after code snippets — which would you rather debug at 11 pm?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi, random

SAFE_CM = 12          # escape trigger
distance
CRUISE = 40           # RPM

def avoid():
    mbot2.straight(-10)
    mbot2.turn(random.randint(60, 120))

while True:
    if cyberpi.ultrasonic2.get() < SAFE_CM:
        avoid()
    else:
        mbot2.forward(CRUISE)
```

Watch out

- `random.randint` needs the import — a good ‘read the error’ moment.
- ‘Why not what’ comments: `# escape trigger distance`, not `# set SAFE_CM to 12`.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Python wander roams the arena reliably
- ✓ `avoid()` extracted; constants named; behavior unchanged
- ✓ Comments explain why, not what

The Wobble, Explained

Goal of this hour: Students understand proportional control conceptually — correction proportional to error.

YOU NEED — PIECES & MATERIALS



Masking tape
taped line on the floor

Classroom



Blindfold
the control-theory theatre

Classroom



Engineering journal
hand computations

Classroom



Printed cards
printed offset scenarios

Classroom

TEACH IT — THE CONCEPT

Resurrect Grade 7's board question: *why does the line follower zig-zag?* Now they can answer it. Bang-bang control knows only three answers (hard left / straight / hard right) — it always overcorrects, overshoots, and corrects back: the wobble is built into the logic. The fix is one idea: **make the correction proportional to the error**. Slightly off → tiny steering nudge; far off → strong steering. In code it will be one line: $\text{steer} = \text{error} \times \text{gain}$.

Teach it with the blindfold walk before any code: one student walks a taped line blindfolded; their partner may only say “left”/“right” (bang-bang) — watch them wobble. Then allow “a little left” / “way left” (proportional) — the walk visibly smooths. Ten minutes of theatre worth a thousand words. Then define terms on the board: **error** (the line offset they measured in Session 3), **gain** (how aggressively we respond), and predict together what too-low gain (drifts off curves) and too-high gain (wobbles again!) will look like tomorrow.

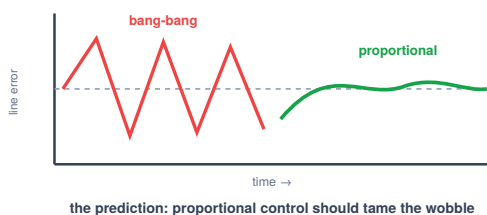
LESSON FLOW (45' CORE)

- 12'** Blindfold line-walk, bang-bang vs. proportional instructions; debrief what changed.
- 12'** Board the model: $\text{error} \rightarrow \times \text{gain} \rightarrow \text{steering}$; connect error to Session 3's offset number.
- 14'** Paper-trace the algorithm on drawn scenarios: offset -60, gain 0.5 → motors? Teams compute by hand.
- 7'** Predictions in the journal: sketch what gain-too-low and gain-too-high trajectories will look like.

Watch out

- Do NOT code today — the concept needs its own hour; tomorrow's implementation then takes minutes.
- The hand computation ($\text{offset} \times \text{gain} \rightarrow \text{two motor speeds}$) is the comprehension gate; don't skip it.

EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Class answered the Grade 7 wobble question in control terms
- ✓ Teams hand-computed motor speeds from offset $\times$ gain correctly
- ✓ Both failure predictions (low/high gain) sketched

P-Control

Goal of this hour: Proportional line following implemented and tuned — the wobble visibly defeated.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Line-following
map

same track all laps

mBot2 box



Stopwatch
lap times

Classroom



Engineering
journal

gain table

Classroom



mBot2 Shield
same battery level
across laps

mBot2 box

TEACH IT — THE CONCEPT

Implementation day: read the offset, multiply by gain, split the result across the two tracks. The core is genuinely three lines — emphasize that a big idea can be small code. Then tuning: start with a low gain, raise it until the robot hugs curves, and keep going until it wobbles again (yesterday's prediction comes true — applaud it, it means the model is right).

Tuning protocol is Session 15/13 discipline: one gain value → one timed lap → log it → change once. Teams produce a small gain table (gain / lap time / observation) and pick their champion value. Vocabulary upgrade: they are tuning a **control loop** — the same mathematics that keeps drones level and cruise control steady. They should know the thing they built today is not a toy idea.

LESSON FLOW (45' CORE)

- 12'** Type the P-follower; first run at gain ≈ 0.3 — expect lazy curve-cutting.
- 22'** Tuning sprints: gain table with 5+ values, timed laps, one change at a time.
- 8'** Champion runs: best gain, three laps, best time recorded next to Grade 7's bang-bang times.
- 3'** Journal: your gain table + the value where wobble returned.

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, cyberpi

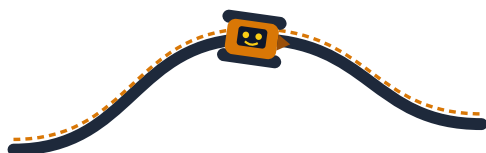
BASE = 45      # cruise RPM
GAIN = 0.5     # tune me!

while True:
    err =
    cyberpi.quad_rgb_sensor.get_offset_track()
    steer = err * GAIN      # the
    whole idea
    mbot2.drive_speed(BASE + steer, BASE -
    steer)
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete.
- If the robot flees the line instantly, the offset sign is inverted — flip the +/- and move on.
- Same track and battery across tuning laps, or the table lies.

● EXPECTED RESULT AT THE END OF THIS SESSION



P-control: the trajectory finally hugs the line

- ✓ P-follower completes laps visibly smoother than bang-bang
- ✓ Gain table with 5+ tested values, one-change discipline
- ✓ Wobble-returns gain found, matching the prediction

Measure It

Goal of this hour: Claims become graphs: error over time proves proportional beats bang-bang.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Line-following
map

mBot2 box



Engineering
journal

graph paper

Classroom



Printed cards
or spreadsheet

Classroom

TEACH IT — THE CONCEPT

Engineers don't say "it looks smoother" — they show a graph. Today: log the error signal while driving (append offset values to a list, or print them and transcribe), then plot error-vs-time for bang-bang and for P-control. The bang-bang plot is a square wave bouncing between extremes; the P-control plot shrinks and settles. One picture, whole argument.

Pair it with lap-time data: P-control is usually both smoother AND faster — a rare free lunch, and worth noticing. Fast teams get the D-term teaser: error $\times$ gain still overshoots a little because it reacts to where the robot IS, not where it's HEADING; adding a term based on how fast error is changing (the derivative) damps that. Even implementing nothing, knowing PID exists — and that they built the P — is the doorway out of this course into real control engineering.

LESSON FLOW (45' CORE)

- 15'** Add error logging to both followers; drive 20 s each, capture the series.
- 15'** Plot both series (paper grid or spreadsheet); overlay or side-by-side.
- 10'** Compare + write the two-sentence evidence-based conclusion under the graphs.
- 5'** D-term teaser for all; extension pairs may prototype it.

PYTHON (MBLOCK → PYTHON MODE)

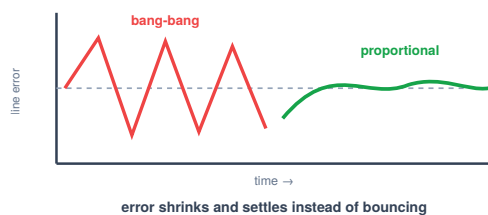
```
errors = []
for i in range(200):          # ~20 s of driving
    err =
    cyberpi.quad_rgb_sensor.get_offset_track()
    errors.append(err)
    steer = err * GAIN
    mbot2.drive_speed(BASE + steer, BASE - steer)
    time.sleep(0.1)

print(errors)                 # copy out
for plotting
```

Watch out

- Transcribing 200 numbers is misery — sample every 5th, or screenshot the console.
- The conclusion sentence must cite the graph ('peak error fell from ~90 to ~25'), not vibes.

EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Both error-time plots produced from real logged data
- ✓ Written conclusion cites numbers from the graphs
- ✓ Team can say what the D in PID would add

Ranger Build I

Goal of this hour: Teardown and the Ranger tank base built — the third and final form begins.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
to tear down

Rover add-on
box



Sorting trays
the ritual

Classroom



Track x2

Rover add-on
box



Wheel hubs
(large + small)

Rover add-on
box



D shaft

Rover add-on
box



Shock absorbers

Rover add-on
box



Rover screws +
nuts

+ socket wrench &
screwdriver

Rover add-on
box



Laptop + mBlock

Ranger guide —
DOWNLOADED

Classroom

TEACH IT — THE CONCEPT

The Ranger is the kit's final form: tracked base plus a servo-driven robotic arm. Today: structured teardown of the Rover (the Grade 8 tray ritual — they know it; hold them to it) and the tank base per the digital guide. Logistics note repeated from the plan: the Ranger guide is **digital-only** — have it downloaded, projected, and ideally printed before class; a dead link kills the hour.

Build quality bar is higher this year: the base will carry a moving arm, so torsional rigidity matters — loose frame screws that were tolerable on the Rover become arm-shake later. The QC checklist gains a line: twist-test the frame gently; nothing should creak or flex visibly.

LESSON FLOW (45' CORE)

- 12'** Rover teardown into trays; electronics padded; parts checked against the Ranger list.
- 28'** Tank base from the guide: front unit (pp. 5–14), then chassis (pp. 15–29); frame screws snugged in a cross pattern.
- 5'** QC + twist test; label robots; journal: what will the arm demand from this base?

QUICK START GUIDE — THIS SESSION

- Guide** mBot2 Ranger Quick Start Guide (the digital ZIP, 61 printed pp.): front unit pp. 5–14 → chassis pp. 15–29. NOT the help-center's "mBot Ranger" — different, older kit
- Screws** front unit opens $\times 8 \times 8 \times 8$; same fastener system as the Rover (M4 $\times$ 10/22/30 + nuts, socket wrench); 3 parts reused from the mBot2 box — read panel counts off the projected guide
- Watch-outs** wheel screws: tighten, then slightly counter-clockwise so the wheel rotates · track grain direction, checked on BOTH sides

Watch out

- Download the guide in advance — digital-only, and school wifi chooses violence on demo days.
- Track tension lesson from Grade 8 applies verbatim; fingerpress test again.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ☒ Teardown complete, trays verified
- ☒ Tank base passes QC + twist test
- ☒ Ranger guide accessible offline for next session

Ranger Build II

Goal of this hour: The arm assembled — and understood as levers, torque and gear ratios in metal.

YOU NEED — PIECES & MATERIALS



Servo pack
the arm's muscle

Rover add-on
box



Brackets 90° /
120° / U

Rover add-on
box



Slide beams +
plates

arm linkage
Rover add-on
box



Boards 1–8
gripper + mounts

Rover add-on
box



Rubber bands
grip traction

Rover add-on
box



Rover screws +
nuts

+ both drivers
Rover add-on
box



Socket wrench

Rover add-on
box



Engineering
journal
safe-range numbers

Classroom

TEACH IT — THE CONCEPT

Assemble the arm: servo, linkage, gripper. Then teach the machine they just built. The arm is a **lever**: the servo twists at the joint, the load hangs at the tip — the longer the arm, the more **torque** the same load demands (why cranes are thick at the base and thin at the tip). If the design routes power through gears: the ratio trades speed for force, like bicycle gears — climbing gear vs. sprint gear.

Make it tactile: unpowered, let students feel the difference between pushing the arm near the joint vs. at the tip. Then the engineering consequence: servos have limits — stall a servo against an obstacle and it overheats. That physical fact becomes next session's software rule: safe ranges, enforced in code. Hardware limits → software constants; that's embedded engineering in one arrow.

LESSON FLOW (45' CORE)

- 25'** Guide pp. 30–61: robotic arm (30–35), grippers ×2 (36–43 — “repeat for the other arm”), finish + cabling (44–61); route cables so full arm travel never pulls them taut.
- 10'** Feel-the-lever stations: joint vs. tip resistance; find the arm's natural range by hand, gently.
- 7'** Board: lever → torque → servo limit → ‘safe range in code’ arrow; write the numbers found by hand.
- 3'** Journal: label a sketch of the arm with joint, load, and where torque is highest.

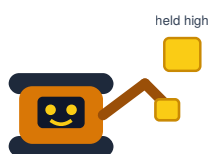
QUICK START GUIDE — THIS SESSION

- Guide** same ZIP: arm pp. 30–35 → grippers pp. 36–43 (build TWO) → finish + cabling pp. 44–61; 3 rear D shafts, then ‘Done!’
- Screws** gripper → servo plate: M4×10 ×4 + nuts ×4 per arm (guide-verified); gripper shells ×1+×2 boards + rubber band per arm
- Cables** STRAIGHT this year: left motor → EM1, right → EM2 — the Rover crossed them, this build does NOT · servo cables → S4 (left) / S3 (right) · short plastic ring first, then long

Watch out

- Students who remember Grade 8 will ‘helpfully’ cross the motor cables — the Ranger wires them STRAIGHT; crossed here means every turn runs backwards.
- Never force the arm past resistance — gear teeth strip silently and are not in the spare bag.
- The hand-found range (e.g. ≈0° floor to ≈100° up) becomes tomorrow's constants — write it down per robot.

● EXPECTED RESULT AT THE END OF THIS SESSION



the arm lifts and holds a block within its safe range

- ✓ Arm assembled; full travel without cable tension
- ✓ Team stated the lever/torque story on their own sketch
- ✓ Physical safe range measured and recorded per robot

First Arm Motions

Goal of this hour: The arm moves under Python control, with hardware limits enforced as named constants.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
built + range
measured

Rover add-on
box



Laptop + mBlock

Classroom



USB cable

mBot2 box



Engineering
journal
why-one-guard
answer

Classroom

TEACH IT — THE CONCEPT

First powered motion: command the arm servo to angles from Python. Start ceremonial — lift, hold, lower, applause — then immediately install the engineering discipline: yesterday's hand-measured range becomes `ARM_FLOOR` and `ARM_TOP` constants, and every arm command goes through a guard function `arm_to(angle)` that clamps requests into the safe range. The robot should be physically unable to hurt itself through code.

This is the session's big idea, bigger than the arm: **defensive programming** — code that assumes mistakes will happen and contains them. Test it adversarially: call `arm_to(500)` on purpose; the guard should quietly deliver `ARM_TOP`. Fast pairs add speed control (move in small steps with sleeps) and discover smooth vs. jerky motion — aesthetics through timing, an echo of Grade 8's animation lesson.

LESSON FLOW (45' CORE)

- 12'** First powered lift/lower; find the servo command in autocomplete; map angle → physical position.
- 18'** Build `arm_to()` with clamping; constants from yesterday's measurements; adversarial tests.
- 10'** Choreography mini-challenge: a 3-pose salute using only `arm_to()` calls.
- 5'** Journal: why should limits live in ONE place (the guard) rather than at every call site?

PYTHON (MBLOCK → PYTHON MODE)

```
import mbot2, time

ARM_FLOOR = 0      # measured by hand,
Session 9
ARM_TOP  = 100

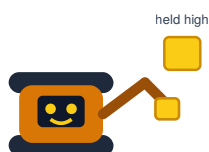
def arm_to(angle):
    # defensive: clamp into the safe range
    angle = max(ARM_FLOOR, min(ARM_TOP,
    angle))
    mbot2.servo_set(angle, 1)  # servo
    port 1

arm_to(80)  # lift
time.sleep(1)
arm_to(500) # attack the guard -> goes to
100
arm_to(10)  # lower
```

Watch out

- Function names below follow mBlock's CyberPi Python API; minor names vary by version — the class rule is: trust the editor's autocomplete. The servo call especially varies — find it together in autocomplete.
- If the arm judders at range edges, back the constants off 5° — mechanical margins are wisdom.

● EXPECTED RESULT AT THE END OF THIS SESSION



the arm lifts and holds a block within its safe range

- ✓ Arm moves to commanded angles via `arm_to()` only
- ✓ Guard proven by adversarial calls
- ✓ 3-pose salute runs smoothly

Pick & Place

Goal of this hour: The gripper earns its keep: lift a block, carry it, stack two — a challenge ladder.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Foam blocks

NOT wooden blocks

Classroom



Masking tape
placement zones

Classroom



Engineering
journal
rung log

Classroom

TEACH IT — THE CONCEPT

The challenge-ladder format: rungs of increasing difficulty, teams climb at their own pace, each rung demoed to the teacher before the next. Rung 1: grip and lift a foam block (grip force/angle tuning — too loose drops, too tight bends). Rung 2: lift, drive 30 cm, set down inside a taped zone (arm + motion coordination, and the discovery that driving shakes the payload — slower is surer). Rung 3: stack a second block on the first (precision: approach angle, release height — drop from too high and the tower falls).

Let the struggle happen — rung 3 is genuinely hard and that's its job. The debug discipline carries: one variable per attempt (grip angle OR approach speed OR release height), logged. Teams should end today knowing their gripper's personality: what it holds well, where it fumbles. That self-knowledge writes Session 12's failure log.

Watch out

- Foam blocks only — wooden ones strip servo gears on a bad grip.
- R2 zone generous (A5-size) today; precision tightens in the capstone.

LESSON FLOW (45' CORE)

- 8'** Ladder briefing + demo protocol; blocks and zones distributed.
- 32'** Climb: grip→lift (R1), carry→place (R2), stack (R3); teacher demos at each rung.
- 5'** Standings + journal: your highest rung and the single change that unlocked it.

● EXPECTED RESULT AT THE END OF THIS SESSION



two blocks stacked inside the target zone

- ✓ Rung 2 (carry & place) reached by every team
- ✓ Rung 3 attempted with logged one-change attempts
- ✓ Gripper strengths/weaknesses written down

Reusable Arm Code

Goal of this hour: pick(), place(), goto() — the arm's verbs, plus an honest failure log.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Foam blocks

Classroom



Engineering
journal

failure log — 20 runs

Classroom

TEACH IT — THE CONCEPT

Yesterday's working sequences become a library: pick() (open, lower, close, lift), place() (lower, open, retreat, lift), goto_cm(d) (drive helper). The capstone mission will then read like English: follow_line() ... pick() ... goto_cm(30) ... place(). When top-level code reads like the mission description, the decomposition is right — that's the design smell to teach.

Second deliverable: the **failure log**. Run pick() twenty times; count successes; describe every failure precisely ('block rotated in grip when arm passed 60°', not 'it dropped it'). An 85% gripper is fine — an *unknown*-% gripper is a time bomb in a scored mission. Reliability engineering in one lunch-sized lesson: measure it, then decide whether to fix it or design around it (e.g. re-grip attempt on failure — which is Session 19's recovery topic, seeded today).

LESSON FLOW (45' CORE)

- 18'** Extract pick(), place(), goto_cm() from yesterday's code; parameters where useful.
- 15'** Failure log: 20 pick() runs, success count + precise failure notes.
- 8'** Library swap-test (the Session 2 ritual): another pair runs your pick() cold.
- 4'** Journal: your reliability %, and fix-it vs. design-around-it — which will you choose?

PYTHON (MBLOCK → PYTHON MODE)

```
def pick():
    grip_open()
    arm_to(ARM_FLOOR + 5)
    grip_close()
    time.sleep(0.3)      # let the grip
    settle
    arm_to(60)           # carry height

def place():
    arm_to(ARM_FLOOR + 10)
    grip_open()
    mbot2.straight(-5)   # retreat before
    lifting
    arm_to(60)
```

Watch out

- The sleep after grip_close matters — servos need a beat; its absence is the ghost bug of the week.
- Failure descriptions must name conditions; 'sometimes' is banned vocabulary.

● EXPECTED RESULT AT THE END OF THIS SESSION

pick() runs: 20
success: 17 (85%)
fail: rotates past 60°
fail: edge grip on tilt
plan: slow lift 60–80°



the failure log — reliability measured, not assumed

- ✓ pick()/place()/goto_cm() pass the swap-test
- ✓ 20-run failure log with precise descriptions
- ✓ A reliability decision made and written down

Navigate + Manipulate *

Goal of this hour: The full pipeline in miniature: follow line → detect marker → stop → pick → deliver.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Line-following
map

mBot2 box



Color patches
the pick marker

Classroom



Foam blocks

Classroom



Masking tape
delivery zone

Classroom

TEACH IT — THE CONCEPT

Integration day — the capstone in miniature, and deliberately so: line following (Session 6) + color-marker detection (Grade 7, Session 16's idea, now in Python) + arm library (Session 12) chained into one mission. The new engineering content is the *seams*: stopping accurately AT the marker (momentum overshoots — the two-speed approach returns yet again), and positioning the arm relative to a block whose location is only as precise as the stop was. Errors compound down a pipeline; students meet that truth physically today.

Run it as a milestone session: the pipeline working once, even slowly and inelegantly, is the win. Teams that get a clean run should immediately answer the harder question: how repeatable? Three runs in a row is the real bar — and the gap between 'worked once' and 'works reliably' is the theme of the entire build-sprint unit ahead.

LESSON FLOW (45' CORE)

- 8'** Whiteboard the pipeline; identify the two seams (stop accuracy, arm alignment).
- 30'** Build and iterate; markers and blocks on the standard course; one-change discipline at the seams.
- 7'** Milestone attempts: pipeline end-to-end; then repeatability count (n clean of 3).

PYTHON (MBLOCK → PYTHON MODE)

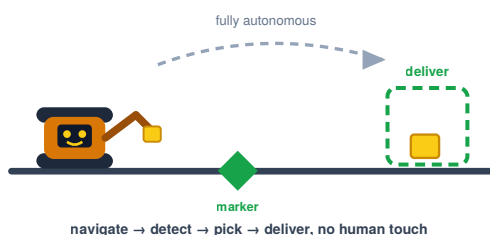
```
while True:
    err =
    cyberpi.quad_rgb_sensor.get_offset_track()
    if
    cyberpi.quad_rgb_sensor.is_color('green'):
        mbot2.EM_stop()
        break
    steer = err * GAIN
    mbot2.drive_speed(BASE + steer, BASE -
    steer)

pick()
mbot2.turn(180)
goto_cm(40)
place()
celebrate()      # Session 2, still earning
its keep
```

Watch out

- Buffer session — a struggling class can pre-place the block at a fixed, known stop point.
- Slow the BASE speed near expected markers; detection at speed is flaky and that's a real sensor lesson.

EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Full pipeline ran end-to-end at least once
- ✓ Both seams named with their observed failure mode
- ✓ Repeatability measured (n of 3), whatever the n

Program Structure

Goal of this hour: The mission code restructured around a main loop + state variable — and paper-traced first.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Engineering
journal

state diagram —
deliverable

Classroom



Printed cards
trace scenarios

Classroom

TEACH IT — THE CONCEPT

Session 13's pipeline was a straight script — fine until something fails mid-mission and the program can't express 'go back and retry'. Introduce the **state machine** pattern: a state variable (SEARCHING / APPROACHING / PICKING / DELIVERING / DONE), a main loop that reads sensors and, depending on the current state, acts and decides transitions. It's Grade 8's mood variable wearing a hard hat — say exactly that; the recognition lands.

Before running: **paper trace**. One teammate is the CPU: finger on the current state, walk through a scenario ('marker seen → state becomes PICKING'), including the nasty ones ('what if the grip fails in PICKING?' → they'll want a retry transition — let them invent it). Tracing catches design holes in minutes that debugging would find in hours; it is also precisely how they'll explain their code in the capstone review.

LESSON FLOW (45' CORE)

- 12'** Board the state diagram for the S13 mission; agree states and transitions as a class.
- 10'** Paper trace in pairs, including two failure scenarios; patch the diagram where it breaks.
- 18'** Restructure the code to the state pattern; behavior should match S13's (refactor discipline!).
- 5'** Journal: your state diagram, photographed or redrawn.

PYTHON (MBLOCK → PYTHON MODE)

```
state = 'SEARCHING'

while state != 'DONE':
    if state == 'SEARCHING':
        follow_line_step()
        if see_marker():
            state = 'PICKING'
    elif state == 'PICKING':
        if pick_ok():
            state = 'DELIVERING'
        else:
            state = 'SEARCHING' # retry
    path!
    elif state == 'DELIVERING':
        goto_cm(40); place()
        state = 'DONE'
```

Watch out

- States are NOUNS-in-progress, transitions are events — keep the diagram honest.
- If the trace can't answer 'what happens if X', the code can't either; fix the diagram first.

● EXPECTED RESULT AT THE END OF THIS SESSION

```
state = 'SEARCHING'
while state != 'DONE':
    if state == 'SEARCHING':
        ...
    elif state == 'PICKING':
        ...retry on fail...
```

```
d = 34 cm
d = 27 cm
d = 12 cm
STOP
```

the mission as a state machine — retry paths included

- ✓ State diagram covers the mission incl. one retry path
- ✓ Paper trace survived two failure scenarios
- ✓ Restructured code matches S13 behavior

Design Sprint

Goal of this hour: Each team's capstone mission fully specified, scoped and signed off.

YOU NEED — PIECES & MATERIALS



Engineering journal
spec: 4 artifacts
Classroom



Ruler / tape measure
course dimensions
Classroom



Printed cards
rubric + example missions
Classroom



Ranger, assembled
feasibility checks only
Rover add-on box

TEACH IT — THE CONCEPT

The brief, in full: design an autonomous mission simulating a real job — warehouse sorting, rescue supply delivery, harvesting, waste collection, or the team's own pitch. The robot must navigate (line or programmed path), detect a target (marker/color), manipulate (pick and deliver at least one object) — fully autonomously. Demo Day (Session 20): scored runs plus a 5-minute pitch that must include one honest failure story.

Today produces four artifacts, in order: **problem statement** (what real job, why it matters — the pitch's backbone); **success criteria** (measurable: 'delivers 2 blocks to zones in <90 s, 2 of 3 runs'); **course sketch** (top-down, dimensions, line path, marker and zone positions); **task breakdown** mapped onto Sessions 1–14 (every task should point at a session where its skill was built — anything pointing nowhere is scope risk, cut it or flag it). The sign-off gate is strictest of all three years: ambition is celebrated, unscoped ambition is returned for revision. 'Impressive and finished' beats 'amazing and imaginary' — write it on the board.

LESSON FLOW (45' CORE)

- 8'** Brief + rubric + example missions pitched fast (steal-able starting points).
- 12'** Problem statement + success criteria drafted; criteria must contain numbers.
- 18'** Course sketch + task breakdown with session references.
- 7'** Sign-off interviews: scope probe ('which task is riskiest? what's your cut line?').

Watch out

- Ban criteria without numbers — 'works well' is not a criterion.
- Require a written cut line: the feature they'll drop first if time runs short. Deciding it now removes the mid-sprint agony.

● EXPECTED RESULT AT THE END OF THIS SESSION

job: pharmacy delivery
succeed: 2 zones < 90 s
course: fig. 8 + 2 markers
riskiest: double pick
cut line: 2nd delivery



the signed mission spec — numbers, sketch, cut line

- ☒ All four artifacts complete; criteria numeric
- ☒ Every task maps to a source session
- ☒ Cut line written; sign-off passed

Build Sprint I *

Goal of this hour: Navigation milestone: the robot completes its course path end-to-end, reliably.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Line-following
map

or taped path

mBot2 box



Masking tape
course build

Classroom



Foam blocks

Classroom



Stopwatch

Classroom



Phone (video/
angle)

photograph the
layout + tape
measure

Classroom

TEACH IT — THE CONCEPT

Sprint rules, borrowed from real teams: a 2-minute standup at the top (each team: done / doing / blocked — blockers get teacher attention first), then heads-down build, then a milestone check at the end. Today's milestone for everyone regardless of mission: **navigation end-to-end** — the robot travels its full course path and stops where the mission needs it, three times in a row.

Hold the line on order: teams itching to build the arm show first that the robot can reliably *arrive*. Navigation is the foundation every later behavior stands on; a wobbly foundation makes every arm bug ambiguous ('did the grip fail or did we stop 4 cm short?'). Reliability before features — it's the sprint's only commandment.

Watch out

- Buffer session: teams behind on P-control tuning catch up here — that IS their navigation work.
- Courses on the floor get shuffled between sessions; photograph the layout with a tape measure in frame.

LESSON FLOW (45' CORE)

- 5'** Standup circle: done / doing / blocked; blockers triaged.
- 33'** Build: course laid out to the sketch's dimensions; navigation tuned on the real course.
- 7'** Milestone check: 3 consecutive end-to-end navigation runs, witnessed and logged.

● EXPECTED RESULT AT THE END OF THIS SESSION



2-minute standup: done · doing · blocked

- ✓ 3 consecutive clean navigation runs on the team's own course
- ✓ Standup done in under 2 minutes with a real blocked-list
- ✓ Course layout photographed for exact rebuild

Build Sprint II + Code Review

Goal of this hour: Manipulation development advances — and every team's code survives a peer review.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Printed cards
review checklist x2
per team
Classroom



Foam blocks
Classroom



Engineering
journal
2+2 review notes
Classroom

TEACH IT — THE CONCEPT

Build continues (arm behaviors onto the navigation spine), but the hour's signature event is the **cross-team code review** — the professional practice, scaled down. Pairs of teams swap laptops and read each other's code against a four-line checklist: *functions with clear names?* · *constants instead of magic numbers?* · *comments that say why?* · *no copy-pasted near-duplicates?* Reviewers write two 'strengths' and two 'suggestions' — the discipline of kind precision.

Frame it before anyone reads a line: review is about the code, not the coder; every suggestion is a gift that saves a future debugging hour. The checklist is the capstone rubric's code-quality section verbatim — so today is a free mock exam. Teams then get sprint time to act on what review found; acting on feedback is the half of the skill that classrooms usually skip.

Watch out

- Model one review comment style first: 'pick() is very readable' / 'GAIN=0.5 appears 3× — constant?'
- A team 'too busy to review' is the team that most needs it — the checklist takes 12 minutes, hold firm.

LESSON FLOW (45' CORE)

- 5'** Standup + review protocol and the four-line checklist.
- 12'** Cross-team review: read, checklist, write 2 strengths + 2 suggestions.
- 23'** Sprint: apply at least one review suggestion, then continue manipulation build.
- 5'** Milestone pulse: where is each team vs. the plan? Cut lines re-checked aloud.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Review exchanged: 2+2 written comments per team
- ✓ At least one suggestion applied the same hour
- ✓ Manipulation visibly progressed on the mission course

Build Sprint III

Goal of this hour: Full mission integration: navigation + manipulation attempts on the complete course.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Foam blocks

Classroom



Engineering
journal

run log on the wall

Classroom



mBot2 Shield

log battery level too

mBot2 box

TEACH IT — THE CONCEPT

Integration week peaks: manipulation joins navigation and full mission attempts begin. Expect the seam bugs from Session 13 at mission scale — stop accuracy vs. arm alignment — plus the new one: **cumulative drift** over a longer course (small navigation errors sum; missions that pick early and deliver late feel it most). The state-machine structure from Session 14 earns its keep now: teams with clean states can add a re-align step to one state without touching the rest.

Keep a visible run log per team on the wall: attempt number, phases completed, failure point. The log turns frustration into data — ‘we fail at delivery 3 of 4 times’ is actionable; ‘it keeps breaking’ is despair. It also quietly writes the failure-story half of the pitch. Teams at their cut line make the cut today, without shame — point at the board from Session 15.

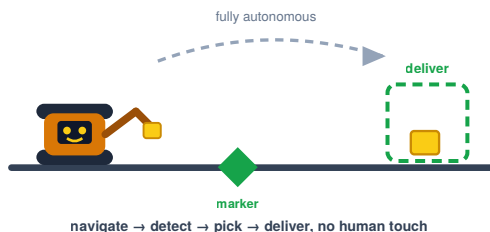
Watch out

- Enforce the log — memory lies under sprint pressure, paper doesn’t.
- Battery sag changes stop distance late in the hour; note battery level in the log too (Grade 7 wisdom, still true).

LESSON FLOW (45’ CORE)

- 5’** Standup; run-log sheets posted per team.
- 33’** Full mission attempts; every attempt logged; fixes between runs follow one-change discipline.
- 7’** Milestone: at least one full-mission success, or the cut executed and the reduced mission succeeding.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Full mission completed at least once (original or post-cut scope)
- ✓ Run log with phase-level failure data
- ✓ One drift/seam fix implemented via a single state change

Rehearsal & Pitch Prep *

Goal of this hour: Failure recovery hardened, run configuration frozen, and the 5-minute pitch rehearsed.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Stopwatch

strict 5:00

Classroom



Engineering
journal

frozen race config

Classroom



Printed cards

pitch structure card

Classroom

TEACH IT — THE CONCEPT

Two workstreams. **Robot:** rehearse under Demo-Day conditions (official start, audience noise simulated, two attempts) and add cheap recovery where the run log points — the classics: re-grip once if pick fails (Session 12's seed sprouts), re-find line if lost (Grade 7's search behavior returns), timeout-and-skip rather than freeze forever. Then freeze the configuration — the Grade 7 race-config ritual, now second nature.

Pitch (5 minutes, strict): the problem (30 s) → the solution & how it works (2') → live demo (90 s) → **one honest failure story** and what it taught (1'). The failure story is mandatory and rubric-weighted — it's where engineering maturity shows, and after three years of debug discipline every team has a good one. Rehearse with a timer; cutting to fit five minutes is itself the exercise. Assign demo roles: driver (starts the run), narrator (talks over it), catcher (handles the robot if chaos wins).

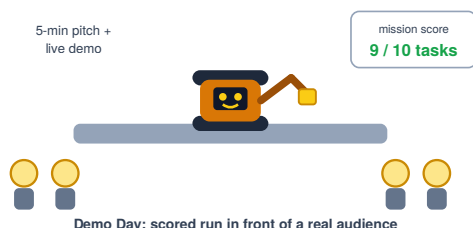
Watch out

- Buffer session: recovery can shrink to timeout-and-skip only — but the freeze and the timed rehearsal are untouchable.
- The failure story must end on the lesson, not the excuse — coach the last sentence.

LESSON FLOW (45' CORE)

- 18'** Recovery pass: implement the top recovery the run log justifies; verify it triggers correctly.
- 8'** Freeze: race configuration written, code committed, no edits after this line.
- 15'** Pitch rehearsal ×2 with timer; peers give one keep / one cut note.
- 4'** Demo-day logistics: order drawn, roles assigned, batteries on chargers.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ One recovery behavior demonstrably triggers and works
- ✓ Configuration frozen in writing
- ✓ Pitch fits 5:00 with all four parts, twice

Demo Day

Goal of this hour: Scored autonomous runs, real pitches, and a three-year retrospective that lands.

YOU NEED — PIECES & MATERIALS



Ranger,
assembled
frozen config

Rover add-on
box



Laptop + mBlock

Classroom



Foam blocks

Classroom



Stopwatch
+ scoreboard

Classroom



mBot2 Shield
charged overnight

mBot2 box



Phone (video/
angle)
course rebuild
photos

Classroom



Printed cards
award certificates

Classroom

TEACH IT — THE CONCEPT

Run it as the event three years earned: audience seated (invite classes, teachers, parents if possible), visible scoreboard (phase points per team's own success criteria + small style bonus), two attempts each with pit time between, pitches delivered from the front. The teacher's job today is master of ceremonies — the students carry the content, and they can.

Close the arc deliberately. The retrospective ritual: the Session 1/ Grade 7 question projected ('what is a robot?') — let them answer it again, now with three years of hands. Then the pathways talk, concrete and short: what they built maps to real fields — control engineering (their P-controller), ML engineering (their mirror), embedded software (their guard functions), robotics competitions, and the kit's 30+ module ecosystem for a robotics club afterlife. End on the honest sentence: they didn't learn about robotics — they did robotics, sixty hours of it.

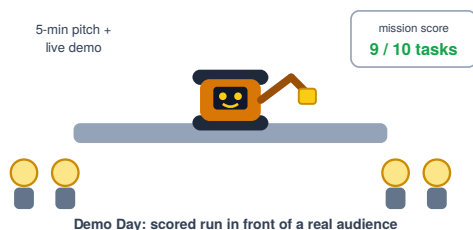
LESSON FLOW (45' CORE)

- 8' Setup: courses rebuilt from photos, scoreboard up, order posted, audience seated.
- 30' The event: per team — pitch (5' incl. live demo) with scored attempts; pit repairs allowed between rounds.
- 5' Awards: precision, engineering, comeback, audience choice.
- 2' Retrospective question + pathways + the closing sentence.

Watch out

- Rebuild courses from the tape-measure photos — 'the course moved' is a heartbreak with a known cure.
- Score against each team's OWN success criteria — missions differ; fairness lives in the spec they signed.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Every team: two scored attempts + a complete 5-minute pitch
- ✓ Failure stories told and applauded
- ✓ Scoreboard archived; three-year arc closed